Introduction to Machine Learning

Lecture 4: Optimisation, Overfitting and Regularisation

Spring semester

Learning as optimisation

- Supervised learning: find parameters w that minimise a loss L(w) on training data
- Empirical risk: L(w) = (1/n) * sum of loss(f_w(x_i), y_i)
- Mean squared error for regression, cross-entropy for classification
- The loss surface can be convex (linear regression) or non-convex (neural networks)

Gradient descent

- Update rule: w <- w - eta * grad L(w)
- eta is the learning rate: too large diverges or oscillates, too small is slow
- On a convex, smooth loss with a suitable eta, gradient descent converges to the global minimum
- On non-convex losses it may stop in a local minimum or saddle point

Stochastic and mini-batch gradient descent

- Full-batch GD uses all n examples per step: accurate but expensive
- SGD uses one example per step: cheap, noisy gradient estimate
- Mini-batch (e.g. 32-256 examples) balances noise and cost, and suits GPUs
- Noise can help escape shallow local minima; learning-rate decay helps convergence

Overfitting and underfitting

- Underfitting: model too simple, high training and test error (high bias)
- Overfitting: model memorises noise, low training error but high test error (high variance)
- Bias-variance trade-off: expected error = bias^2 + variance + irreducible noise
- Diagnose with learning curves: compare training vs validation error

Train, validation and test sets

- Training set: fit parameters. Validation set: choose hyperparameters. Test set: final estimate only
- Never tune on the test set; it leaks information and gives optimistic results
- k-fold cross-validation: rotate the validation fold when data is scarce

Regularisation

- Add a penalty to the loss: L_reg(w) = L(w) + lambda * Omega(w)
- L2 (ridge): Omega = ||w||^2, shrinks weights smoothly towards zero
- L1 (lasso): Omega = ||w||_1, drives some weights exactly to zero -> feature selection
- lambda controls the trade-off: larger lambda = simpler model, more bias, less variance

Other ways to fight overfitting

- Early stopping: stop training when validation error starts rising
- More data and data augmentation
- Dropout in neural networks: randomly drop units during training

Worked example: house prices

- Polynomial regression of degree 1, 3 and 15 on 20 houses
- Degree 1 underfits, degree 15 fits training points exactly but generalises poorly
- Degree 15 with L2 regularisation (lambda = 0.1) recovers a smooth curve

Summary

- Learning = minimising a loss; gradient descent and its stochastic variants do the work
- Generalisation, not training error, is the goal
- Use validation data and regularisation to control model complexity